

IPL CRICKET ANALYSIS & PREDICTION SYSTEM

1. Introduction

This project focuses on analysing IPL cricket match data and finding useful patterns from it. Both match-level and ball-by-ball data were used. The main focus was to understand team and player performance across seasons and to build a simple machine learning model for predicting the winner of a match.

2. Objectives

- Clean and prepare the IPL match and delivery datasets.
- Study team-wise, player-wise and season-wise performance.
- Analyse high-scoring innings, Super Over-type records and umpire trends.
- Identify leading run scorers and wicket takers for each season.
- Build a machine learning model to predict the winner of an IPL match.
- Evaluate the model using accuracy, classification report and confusion matrix.

3. Dataset Details

Two CSV files were used: matches.csv and deliveries.csv. The matches dataset contains 1,095 match records and 20 columns. The deliveries dataset contains 260,920 ball-by-ball records and 17 columns.

The match data includes season, teams, toss details, venue, umpires and match winner. The delivery data includes batting team, bowling team, batter, bowler, runs, extras and wickets.

The data covers IPL seasons from 2007/08 to 2024, with 19 team names and 58 venues.

4. Data Preprocessing

The data was first loaded into Pandas and checked for missing values, duplicate rows, data types and basic inconsistencies. The match date was converted to a proper date format and duplicate records were removed where applicable.

Missing values in numeric delivery fields were handled using 0 where that represented no recorded value. Basic checks were performed after cleaning.

For season-wise player analysis, season information from matches was merged with deliveries using match ID.

5. Exploratory Data Analysis

Match Outcomes

There were 1,095 matches in the dataset. Of these, 1,090 had a decided winner. The remaining results included 14 ties and 5 no-result matches.

Team-wise Performance

Mumbai Indians recorded the highest number of wins with 144, followed by Chennai Super Kings with 138 and Kolkata Knight Riders with 131. Royal Challengers Bangalore recorded 116 wins.

Toss Analysis

Among the 1,090 decided matches, the toss winner also won the match 554 times, or about 50.83%. This shows that winning the toss alone did not guarantee a match win.

High-scoring Innings

The analysis found 211 innings with scores of 200 or more. These innings were used to examine high-scoring performances.

Super Over Analysis

The delivery data did not contain a separate `is_super_over` column. Extra innings were therefore identified using innings numbered 3 or higher. This gave 14 matches with extra-innings records for analysis.

Top Batsmen

V Kohli was the highest run scorer in the analysed delivery data with 8,014 runs, followed by S Dhawan with 6,769 and RG Sharma with 6,630.

Top Bowlers

Run outs, retired hurt and obstructing-the-field dismissals were excluded from the bowler wicket count. YS Chahal was the leading wicket taker with 205 wickets, followed by PP Chawla with 192 and DJ Bravo with 183.

Orange Cap and Purple Cap

Season-wise batting leaders were calculated from the delivery data. Notable totals include V Kohli's 973 runs in 2016, JC Buttler's 863 in 2022 and Shubman Gill's 890 in 2023.

For bowling, DJ Bravo took 32 wickets in 2013, K Rabada took 32 in 2020/21 and HV Patel took 32 in 2021.

Umpire Trends

The two on-field umpire columns were combined to count appearances. AK Chaudhary and S Ravi had the highest counts, with 131 appearances each.

6. Season-wise Analysis

The team with the most wins was identified for every season. Rajasthan Royals led 2007/08 with 13 wins, Mumbai Indians led 2013 with 13, Gujarat Titans led 2022 with 12, and Kolkata Knight Riders led 2024 with 11.

7. Predictive Modelling

A Random Forest Classifier was used for match winner prediction. The target was whether Team 1 won the match.

The model used season, Team 1, Team 2, toss winner and toss decision as input features. Categorical features were converted using OneHotEncoder before being passed to the Random Forest model.

The modelling dataset contained 1,090 decided matches. Team 1 won 555 matches and Team 2 won 535. The data was split into 80% training and 20% testing data, giving 872 training records and 218 testing records.

8. Model Evaluation

The Random Forest model achieved 52.29% accuracy on the test set. The F1-score was 0.49 for Team 2 wins and 0.55 for Team 1 wins.

The confusion matrix showed 50 correctly predicted Team 2 wins and 64 correctly predicted Team 1 wins. There were 57 Team 2 wins predicted as Team 1 wins and 47 Team 1 wins predicted as Team 2 wins.

The accuracy is relatively low, so this model should not be treated as a highly reliable prediction system. It mainly shows that the basic match and toss-related features used here have limited predictive power.

9. Sample Prediction

As a sample test, the model was given a match involving Royal Challengers Bangalore as Team 1 and Kolkata Knight Riders as Team 2. The model predicted Kolkata Knight Riders as the winner.

10. Conclusion

The IPL dataset was successfully cleaned, explored and analysed using Python. The project covered match outcomes, team performance, player statistics, season-wise comparisons, high-scoring innings, Super Over-type records, umpire trends and Cap winners.

A Random Forest model was also built for winner prediction. It achieved 52.29% accuracy with the selected features. Although this is not a strong prediction accuracy, it provides a useful starting point for the machine learning part of the project.

11. Future Improvements

- Add venue information and team performance statistics as model features.
- Include recent form, batting strength and bowling strength.
- Compare Random Forest with Logistic Regression and other suitable models.
- Test different feature combinations and tune model parameters.
- Build a simple interface where match details can be entered for prediction.

12. Technologies Used

- Python
- Pandas
- NumPy
- Matplotlib
- Seaborn
- Scikit-learn
- Jupyter Notebook